МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-68

Выполнил
А. В. Венско,
студент группы АС-68

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 2

Задание. выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики.

Вариант 2

- **Регрессия (Прогнозирование медицинских расходов)**
  1. Medical Cost Personal Datasets
  2. Предсказать страховые выплаты (charges)
  3. **Задания:**
     - загрузите и обработайте категориальные признаки (например, sex, smoker);
     - обучите модель **линейной регрессии** для предсказания charges;
     - рассчитайте **MAE** (Mean Absolute Error) и **R^2**;
     - визуализируйте зависимость charges от bmi (индекс массы тела) с помощью диаграммы рассеяния и линии регрессии.
- **Классификация (Диагностика заболеваний сердца)**
  1. Heart Disease UCI
  2. Предсказать наличие у пациента болезни сердца (target)
  3. **Задания:**
     - загрузите данные и разделите их на обучающую и тестовую выборки;
     - обучите модель **логистической регрессии**;
     - оцените модель с помощью **Accuracy**, **Precision**, **Recall** и **F1-score**;
     - постройте **матрицу ошибок**.

Выполнение:

Код программы

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    mean_absolute_error,
    r2_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
```

```
    confusion_matrix,
    ConfusionMatrixDisplay,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

BASE_DIR = Path(__file__).resolve().parent
INSURANCE_FILE = BASE_DIR / "medical_cost_personal_dataset.csv"
HEART_FILE = BASE_DIR / "heart_disease_uci.csv"

def check_file(file_path):
    if not file_path.exists():
        raise FileNotFoundError(
            f"Не найден файл: {file_path.name}. "
            "Поместите CSV-файл в одну папку с variant2_ml.py."
        )

def regression_task():
    print("\nРЕГРЕССИЯ: ПРОГНОЗИРОВАНИЕ МЕДИЦИНСКИХ РАСХОДОВ ")

    check_file(INSURANCE_FILE)
    data = pd.read_csv(INSURANCE_FILE)
    print("Размер датасета:", data.shape)

    categorical_features = ["sex", "smoker", "region"]
    numerical_features = ["age", "bmi", "children"]

    X = data[categorical_features + numerical_features]
    y = data["charges"]

    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore")),
    ])
    numerical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
    ])

    preprocessor = ColumnTransformer([
        ("categorical", categorical_pipeline, categorical_features),
        ("numerical", numerical_pipeline, numerical_features),
    ])

    model = Pipeline([
        ("preprocessor", preprocessor),
        ("regression", LinearRegression()),
    ])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("\nМетрики качества линейной регрессии:")
    print("MAE:", round(mean_absolute_error(y_test, predictions), 2))
    print("R2:", round(r2_score(y_test, predictions), 4))

    bmi_model = LinearRegression()
    bmi_model.fit(data[["bmi"]], y)

    bmi_values = np.linspace(data["bmi"].min(), data["bmi"].max(), 200)
    charges_line = bmi_model.predict(pd.DataFrame({"bmi": bmi_values}))

    plt.figure(figsize=(9, 6))
    plt.scatter(data["bmi"], y, alpha=0.6, label="Фактические значения")
```

```
    plt.plot(bmi_values, charges_line, linewidth=2, label="Линия регрессии")
    plt.title("Зависимость медицинских расходов от BMI")
    plt.xlabel("BMI (индекс массы тела)")
    plt.ylabel("Charges (медицинские расходы)")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(BASE_DIR / "regression_bmi.png", dpi=300)
    plt.show()
    plt.close()

def classification_task():
    print("\nКЛАССИФИКАЦИЯ: ДИАГНОСТИКА ЗАБОЛЕВАНИЙ СЕРДЦА")

    check_file(HEART_FILE)
    data = pd.read_csv(HEART_FILE)
    print("Размер датасета:", data.shape)
    data["target"] = (data["num"] > 0).astype(int)

    categorical_features = [
        "sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"
    ]
    numerical_features = ["age", "trestbps", "chol", "thalch", "oldpeak"]
    feature_columns = categorical_features + numerical_features

    X = data[feature_columns]
    y = data["target"]

    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore")),
    ])
    numerical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    preprocessor = ColumnTransformer([
        ("categorical", categorical_pipeline, categorical_features),
        ("numerical", numerical_pipeline, numerical_features),
    ])

    model = Pipeline([
        ("preprocessor", preprocessor),
        ("classification", LogisticRegression(max_iter=2000)),
    ])

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y,
    )

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("\nМетрики качества логистической регрессии:")
    print("Accuracy:", round(accuracy_score(y_test, predictions), 4))
    print("Precision:", round(precision_score(y_test, predictions, zero_division=0),
4))
    print("Recall:", round(recall_score(y_test, predictions, zero_division=0), 4))
    print("F1-score:", round(f1_score(y_test, predictions, zero_division=0), 4))

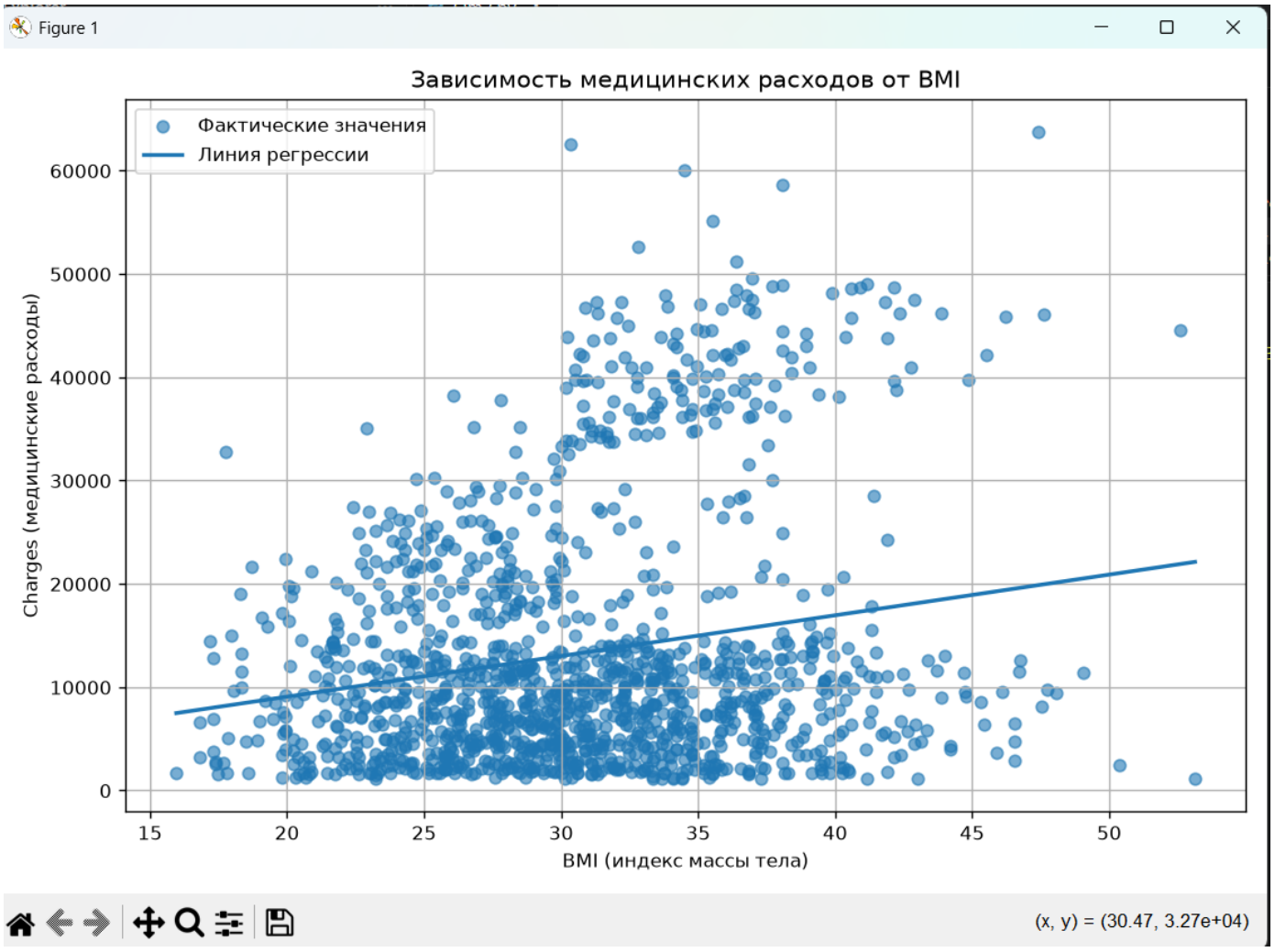
    matrix = confusion_matrix(y_test, predictions, labels=[0, 1])
    display = ConfusionMatrixDisplay(
        confusion_matrix=matrix,
```

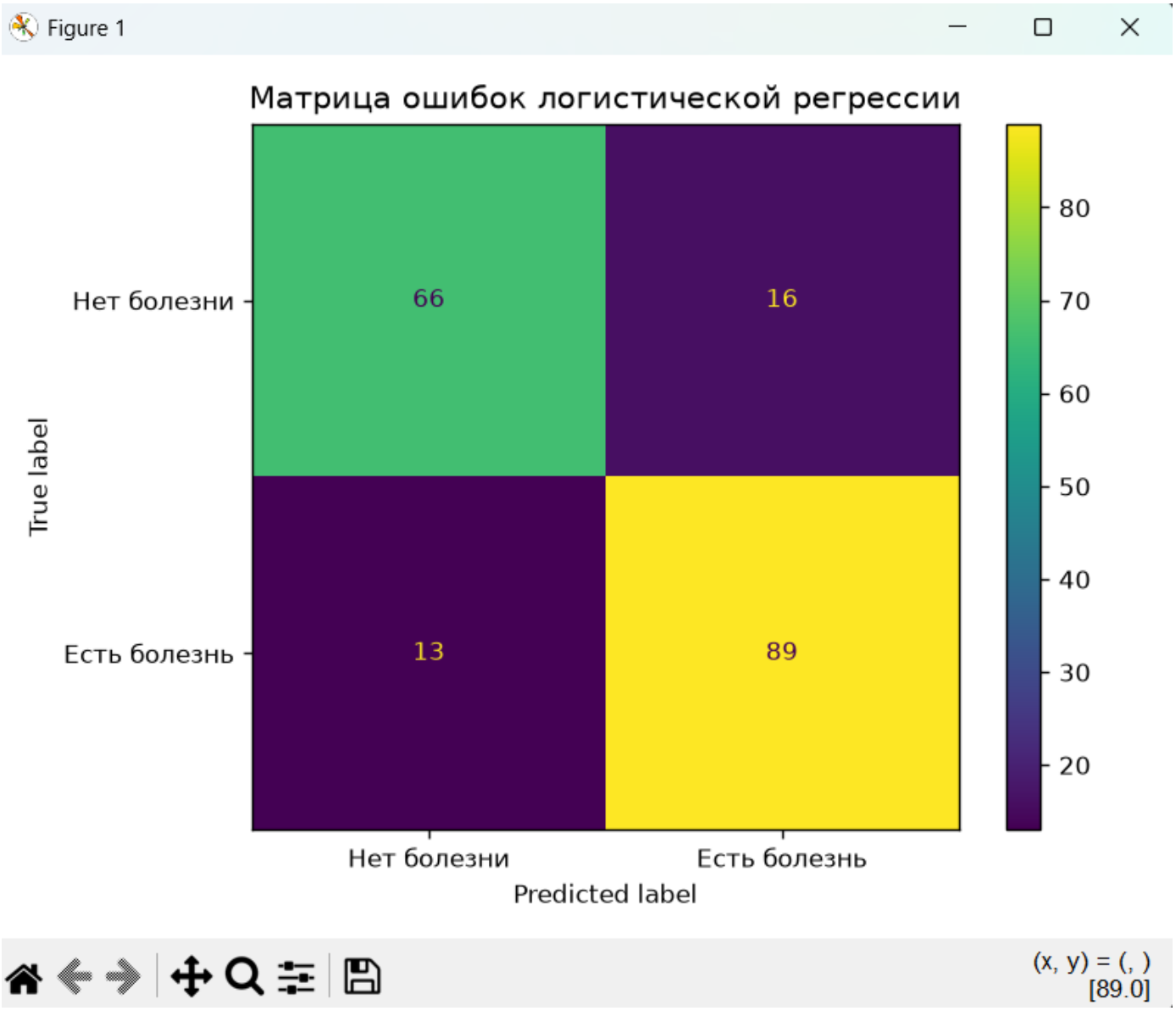
```
        display_labels=["Нет болезни", "Есть болезнь"],
    )
    display.plot(values_format="d")
    plt.title("Матрица ошибок логистической регрессии")
    plt.tight_layout()
    plt.savefig(BASE_DIR / "confusion_matrix.png", dpi=300)
    plt.show()
    plt.close()

def main():
    regression_task()
    classification_task()
    print("\nРабота завершена.")

if __name__ == "__main__":
    main()
```

Результат:





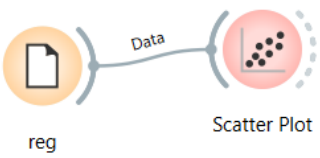
РЕГРЕССИЯ: ПРОГНОЗИРОВАНИЕ МЕДИЦИНСКИХ РАСХОДОВ
Размер датасета: (1338, 7)

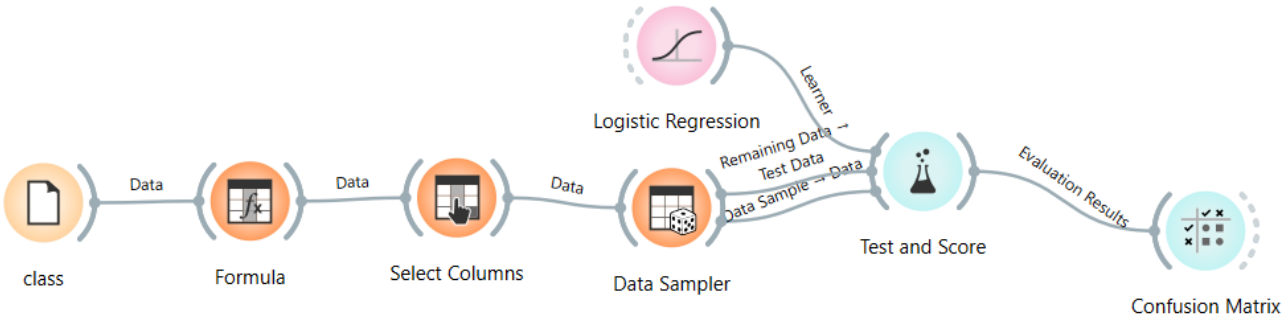
Метрики качества линейной регрессии:
MAE: 4181.19
R2: 0.7836

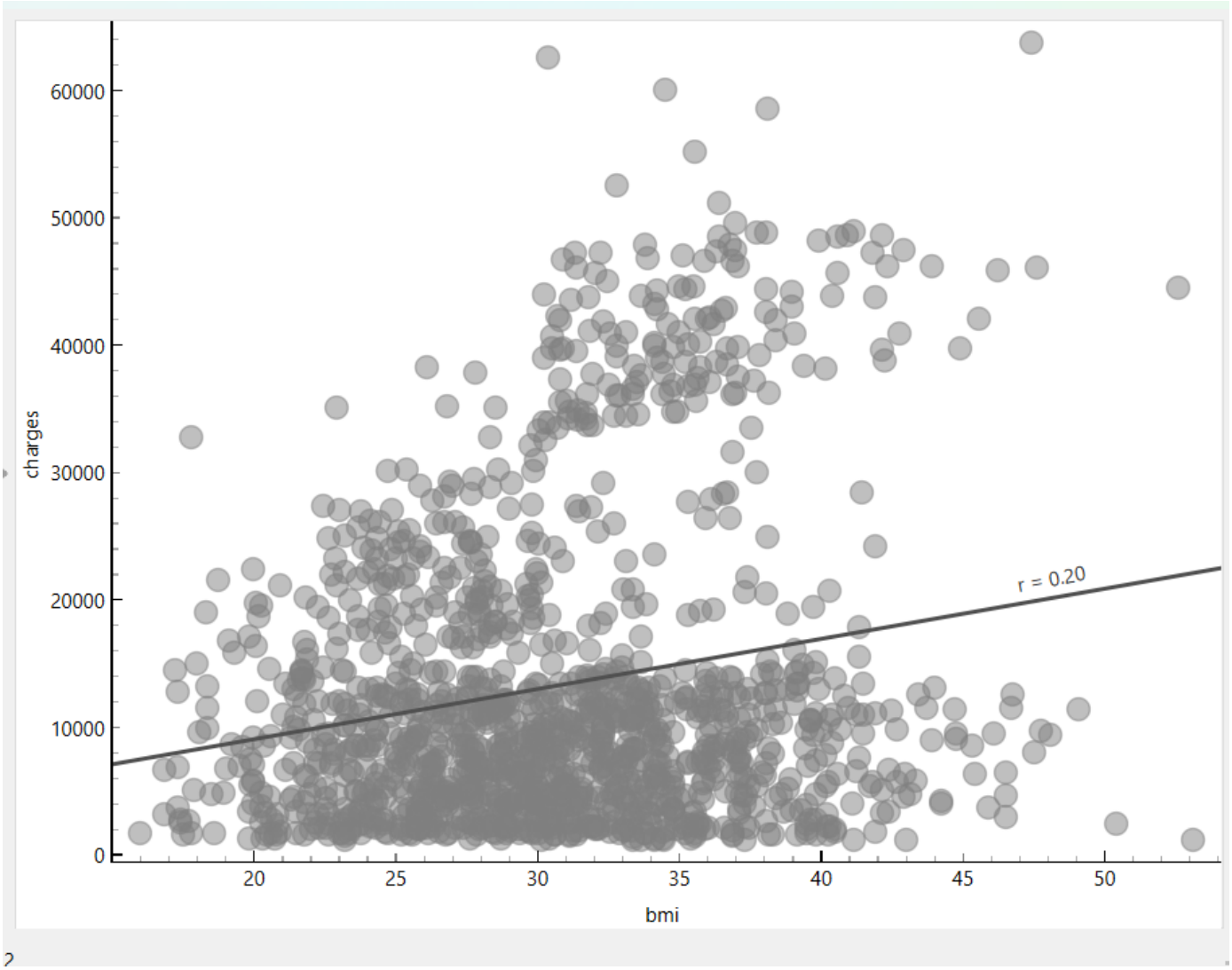
КЛАССИФИКАЦИЯ: ДИАГНОСТИКА ЗАБОЛЕВАНИЙ СЕРДЦА
Размер датасета: (920, 16)

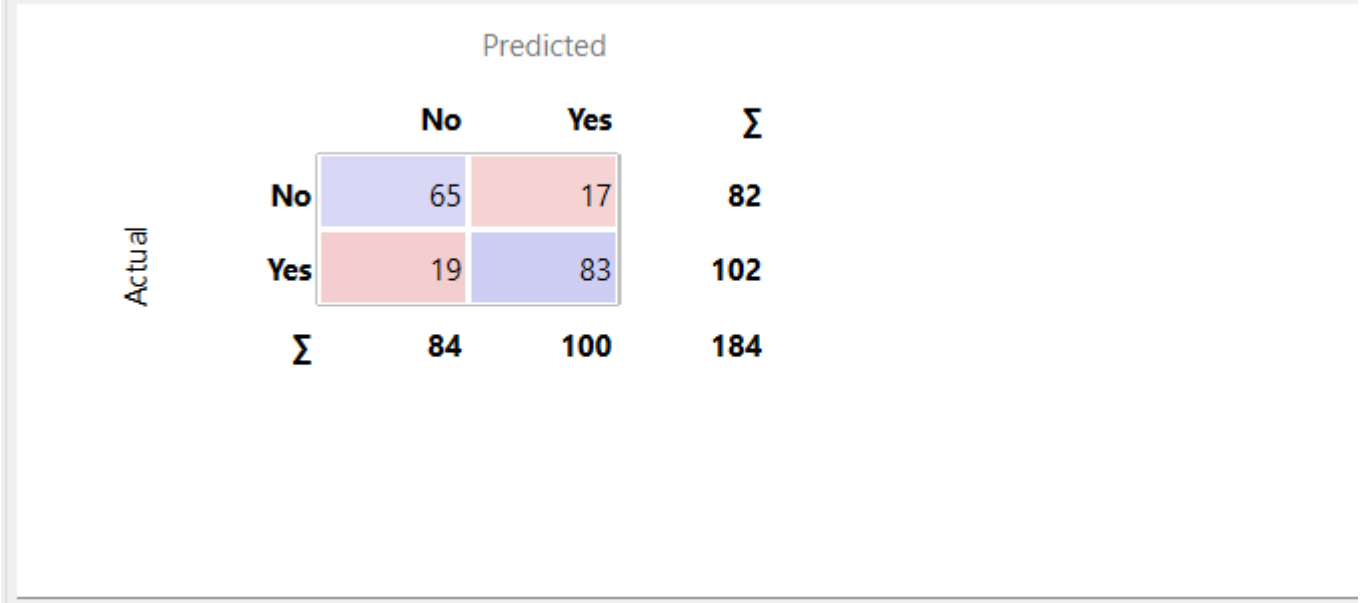
Метрики качества логистической регрессии:
Accuracy: 0.8424
Precision: 0.8476
Recall: 0.8725
F1-score: 0.8599

Результаты в Orange:


Data
reg
Scatter Plot


Logistic Regression
Learner
Remaining Data
Test Data
Data Sample
Data
Data
Data
Data
Evaluation Results
class
Formula
Select Columns
Data Sampler
Test and Score
Confusion Matrix


60000
50000
40000
30000
20000
10000
0
charges
r = 0.20
20
25
30
35
40
45
50
bmi



Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.